

Results

Fixture Release Packet I

The fixture release packet contains fourteen segments spanning four classification levels: UNCLASSIFIED (ten segments), CUI (one segment), SECRET (two segments), and TOP_SECRET (one segment). Four segments carry source controls (drawn from the HUMINT, SIGINT, and IMINT disciplines). Twenty-one redaction decisions are applied across those four segments, using all five bounded reasons: `source_identity`, `operational_detail`, `time_place_selector`, `legal_privilege`, and `privacy`.

The audit produces:

- ▶ **Releasable**: true (no error-level findings after redaction).
- ▶ **Release safety score**: weighted by error count, warning count, and mosaic risk.
- ▶ **Redaction coverage**: 1.0 (all sensitive segments have at least one decision).
- ▶ **Mosaic risk score**: residual markers normalized by segment and pattern count.

Fixture Release Packet II

- ▶ **Findings:** warning-level findings for residual markers in sanitized text.

Source-Safe Redaction Ledger I

The redaction ledger records each decision with:

- ▶ A decision ID derived from the SHA-256 of the segment ID, span, reason, and replacement.
- ▶ The segment ID, start, end, span length, reason, and replacement.
- ▶ A `valid_span` flag indicating whether the span falls within the segment text.
- ▶ A `source_span_sha256` hash of the redacted text—present only for valid spans, absent for invalid or orphan decisions.

The ledger never exposes source text. It provides reproducible audit evidence without disclosure risk.

Segment Hash Manifest I

The hash manifest records, for each segment:

- ▶ `source_sha256`: SHA-256 of the original segment text.
- ▶ `public_sha256`: SHA-256 of the redacted segment text.

This allows downstream verification that the public release matches the audited version without comparing source text directly.

Review Gate I

The release gate requires three reviewer roles: originator, classification reviewer, and release authority. Each reviewer provides a non-empty rationale. The fixture reviews all approve, yielding:

- ▶ **Approved:** true.
- ▶ **Approval count:** 3.
- ▶ **Required roles present:** classification_reviewer, originator, release_authority.
- ▶ **Findings:** none.

The `final_release_recommended` flag is true only when the packet is releasable, the review gate is approved, and no blocking warnings remain.

Visual Proof Matrix I

The development proof matrix produces sixteen base PDFs (four redaction styles $\times$ four backgrounds) and, when steganography is enabled, sixteen companion steganography PDFs with hash manifests. Each variant records:

- ▶ Base PDF filename, byte size, and SHA-256.
- ▶ Steganography PDF filename, byte size, and SHA-256.
- ▶ Hash manifest filename and SHA-256.
- ▶ Kmyth sidecar count and filenames (when Kmyth is available).

Kmyth TPM Sidecar Production I

When Kmyth tools are runnable and a TPM backend is available, each variant produces two `.ski` sidecars:

1. `{variant_id}.hashes.json.ski` — the hash manifest sealed against the TPM storage hierarchy.
2. `{variant_id}_steganography.pdf.ski` — the steganography PDF sealed against the TPM storage hierarchy.

The `.ski` files are ASCII-armored and contain PCR selections, policy or-values, the storage key public area, and the sealed data object. Unsealing requires the same TPM and platform configuration.

In the verified run, all sixteen variants produced both sidecars, yielding thirty-two `.ski` files total. The `kmyth-seal` binary's `FlushContext` patch ensured that consecutive seal invocations did not exhaust the `swtpm` transient object slots.

Residual Risk Detection I

The residual-risk detector scans sanitized text for common public-release leaks:

Pattern	Regex
Email address	\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b
IPv4 address	\b(?:\d{1,3}\.){3}\d{1,3}\b
Coordinate pair	\b-?\d{1,2}\.\d{3,},\s*-?\d{1,3}\.\d{3,}\b
Controlled dissemination	\b(?:NOFORN ORCON REL\s+TO)\b
Collection discipline	\b(?:HUMINT SIGINT IMINT MASINT OSINT)\b
Compartment marker	\b(?:SCI TS_SCI TOP\s+SECRET)\b
Sensitive markers	HUMINT, SIGINT, source, selector, location, 2026-

Residual Risk Detection II

Each detected marker generates a warning finding. The mosaic risk score aggregates residual markers across all segments.

Discussion

Discussion I

The exemplar demonstrates that disclosure control can be decomposed into orthogonal concerns: text-level audit, visual presentation, steganographic provenance, and hardware-backed sealing. Each concern is independently configurable and verifiable.

The visual proof matrix confirms that blackout, whiteout, grayout, and blur treatments produce equivalent source-safe outputs—only the visual token differs. The steganography layer adds provenance without altering the redaction decisions. Kmyth TPM sealing adds a hardware binding that ensures sealed sidecars can only be unsealed on the same platform configuration.

The mssim-to-swtpm protocol proxy is a necessary bridge on macOS, where no hardware TPM exists. The proxy handles three protocol mismatches: (1) platform commands use different command numbers, (2) the data channel wraps TPM commands in a 9-byte mssim header, and (3) swtpm's transient object slots are limited and must be flushed between seal invocations.

Discussion II

The FlushContext patch to kmyth-seal is a critical fix for batch sealing workflows. Without it, the second kmyth-seal invocation fails with “out of memory for object contexts” because the storage key from the first invocation remains loaded in the TPM's transient object table.

Limitations I

- ▶ The fixture data is invented; real release packets may require additional classification taxonomies and review-role policies.
- ▶ The swtpm software TPM does not provide hardware-level tamper resistance; production deployments should use a hardware TPM.
- ▶ The mssim proxy adds latency to each TPM command; batch sealing of thirty-two sidecars takes approximately thirty seconds.
- ▶ PDF password encryption is optional and uses AES-256; the password is not stored in the variant matrix.